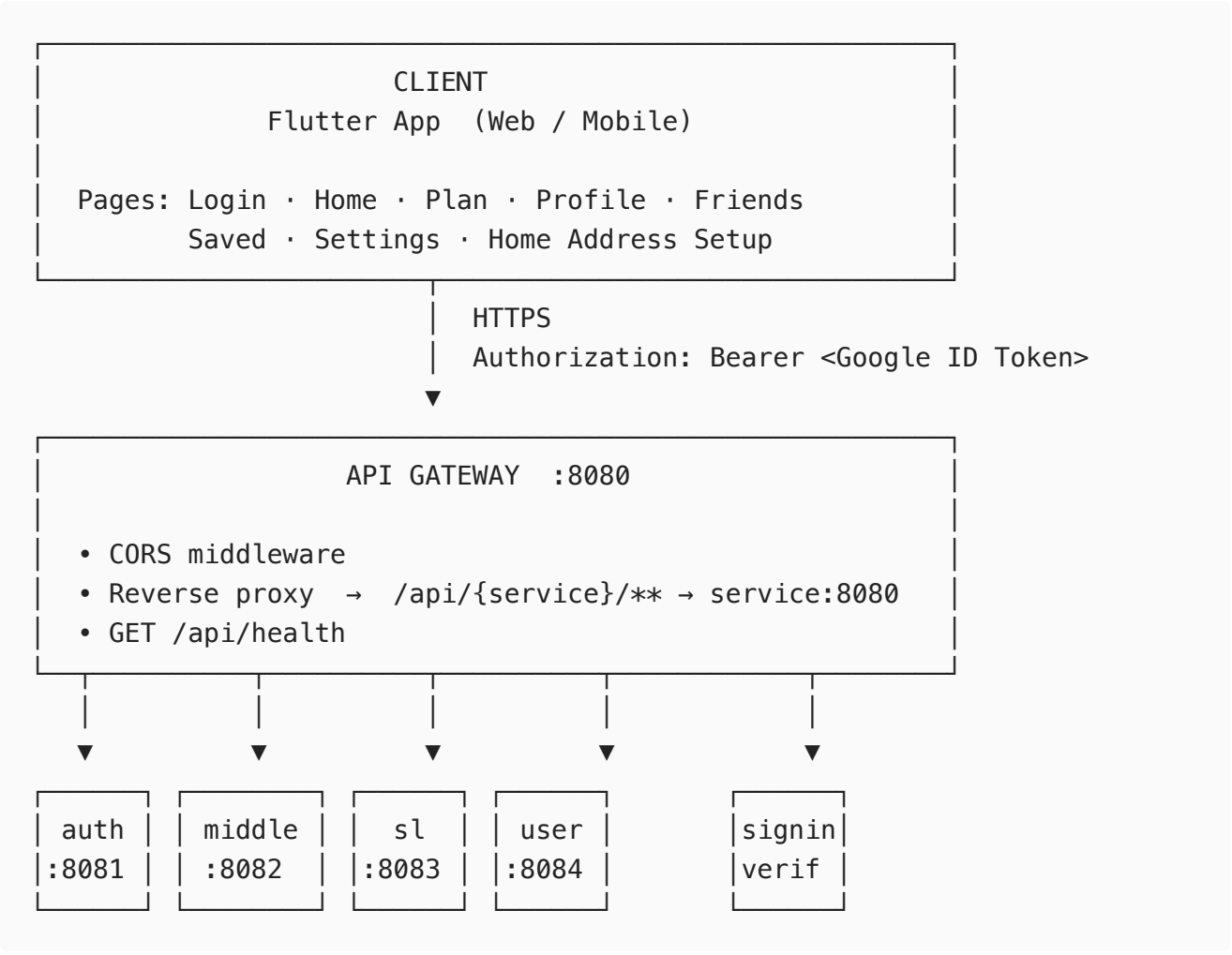
Architecture

System Architecture

Overview

A Flutter frontend backed by a Go microservices architecture. The API Gateway acts as the single entry point, reverse-proxying all traffic to internal services. Services share a common router, auth middleware, and PostgreSQL database.

High-Level Diagram



Services

Shared Infrastructure (`backend/shared/`)

All services (except `api-gateway`) are bootstrapped via `standardrouter.Init()`, which sets up:

```
standardrouter.Init()
  ├── Gorilla Mux router
  ├── Route prefix:  /api/{SERVICE_NAME}/v1/
  ├── IP rate limiter  (10 req/s per IP, auto-cleanup after 3 min idle)
  ├── GET /health  (open)
  └── /auth/** subrouter  ──►  AuthMiddleware
                                  └── Validates Google ID Token (Bearer)
                                      Checks audience against known client
IDs
                                      Injects UserClaims into request
context
```

API Gateway (`backend/api-gateway/`) — port 8080

```
Startup
  └── Reads  ../services/  directory
  └── Registers  /api/{name}/**  →  http://{name}:8080  (reverse proxy)

Middleware stack
  └── CORS  (Access-Control-Allow-Origin: *)
```

User Service (`backend/services/user/`) — port 8084

```
Database: PostgreSQL (shared/db)

Open endpoints
  POST  /api/user/v1/login                ← Google Sign-In entry point

Auth-required endpoints  (/api/user/v1/auth/...)
  GET   /search                           ← search users
  POST  /friends/request                  ← send friend request
  GET   /friends                          ← list friends
  GET   /friends/pending                  ← list pending requests
  PUT   /friends/{id}/accept
  PUT   /friends/{id}/decline
  PUT   /home-address                     ← set user home address

Internal packages
  controller/  login · profile · friends · search
  repository/  user_repo · friend_repo
  googleauth/  Google OAuth helpers
```

Middle Service (`backend/services/middle/`) — port 8082

```
Open endpoints
  POST  /api/middle/v1/middleplaces        ← core feature endpoint

Core logic
  middle/
    Average()   — arithmetic mean of N coordinates
    Median()    — median of N coordinates

  graph/
    NewWithData()    — loads SL transit graph on startup (cached in
memory)
    slPointSearch()  — calls SL Journey Planner API for transit time
    calculate.go     — Dijkstra / shortest-path over the SL graph

  places/
    NearbySearch()   — Google Places API  (primary)
    Overpass/OSM     — fallback for opening hours, phone, website, cuisine

Flow:  receive N locations  →  compute midpoint  →  build SL graph
subgraph
       →  find transit-optimal meeting point  →  fetch nearby places  →
return
```

SL Service (`backend/services/sl/`) — port 8083

```
Open endpoints
  GET   /api/sl/v1/trip    ?from=<Address JSON>&to=<Address JSON>
  ANY   /api/sl/v1/trips   (controller.TripEndpoint)

Internal packages
  searchaddress/   AddressSearch()  — queries SL API for address-based
trips
  controller/      TripEndpoint
```

Sign-In Verification Service (`backend/services/signinverification/`)

```
Open endpoints
  POST  /api/signinverification/v1/verify-google-signin
```

```
Internal packages
  verifyer/     googleverifyer.go  — verifies Google ID token
  controller/   VerifyGoogleSignInEndpoint
```

Auth Service (`backend/services/auth/`) — port 8081

```
Open endpoints
  GET  /api/auth/health
  GET  /api/auth/example    → {"message": "hello from example"}

Status: placeholder / reserved for future auth flows
```

Database

```
PostgreSQL 16  (docker: db, port 5432)
  POSTGRES_DB:    mydb
  POSTGRES_USER:  user

  Persistent volume:  db-data
  Health check:       pg_isready -U user -d mydb

  Consumers
    └── user service  (users, friends tables via shared/db + sqlx)
```

External APIs

```
Google
  ├── Google Sign-In       ← frontend (Flutter google_sign_in)
  ├── ID Token Validation  ← signinverification + shared/router
AuthMiddleware
  └── Places API           ← middle service  (places.googleapis.com)

SL (Storstockholms Lokaltrafik)
  └── Journey Planner API  ← middle service + sl service
      (journeyplanner.integration.sl.se/v2/trips)

OpenStreetMap / Overpass
  └── Overpass API         ← middle service (fallback for place metadata)
      overpass-api.de  |  overpass.kumi.systems  |
overpass.openstreetmap.ru
```

```
Mapbox
  └── Geocoding API        ← frontend (MapboxGeocodingService.dart)
```

Auth Flow

```
1. User taps "Sign in with Google"
         │
         ▼
2. Flutter  →  Google Sign-In SDK  →  Google ID Token
         │
         ▼
3. POST /api/signinverification/v1/verify-google-signin
         │   { idToken: "..." }
         ▼
4. signinverification validates token via Google API
         │
         ▼
5. POST /api/user/v1/login
         │   (stores/retrieves user, returns session info)
         ▼
6. All subsequent calls include:
         Authorization: Bearer <Google ID Token>
         │
         ▼
7. api-gateway proxies to service
         │
         ▼
8. shared/router AuthMiddleware validates token
   extracts { GoogleID, Email, Name } → request context
```

Docker Compose Services

Service	Image/Build	Host Port	Internal Port
api-gateway	DockerGateway	8080	8080
auth	DockerService	8081	8080
middle	DockerService	8082	8080
sl	DockerService	8083	8080
user	DockerService	8084	8080
db (postgres:16)	official image	5432	5432

Production variants: `DockerGateway.prod`, `DockerMiddle.prod`, `DockerService.prod`

Repository Structure

```
pvt-app/
├── backend/
│   ├── api-gateway/         gateway reverse proxy
│   ├── services/
│   │   ├── auth/            placeholder auth service
│   │   ├── middle/          midpoint + transit + places logic
│   │   ├── sl/              SL trip search
│   │   ├── signinverification/  Google token verification
│   │   └── user/            user profiles + friends
│   └── shared/
│       ├── db/              PostgreSQL connection (sqlx)
│       ├── models/          location.Point, location.Address, ...
│       └── router/          standardrouter: mux, rate limiter, auth
middleware
├── frontend/                Flutter app
│   └── lib/
│       ├── pages/           login · home · plan · profile · friends · ...
│       ├── services/        auth_service · friends_service ·
location_service
│       └── models/
├── doc/                     architecture docs
├── docker-compose.yml
├── Makefile
└── go.mod                   module: github.com/durelius/pvt-app
```